

ENGENHARIA DE SOFTWARE

REVISÃO

Gilberto Falco Netto

Tecnólogo em Análise e Desenvolvimento de Sistemas,
Faculdade Anhanguera - Jundiaí/SP

Qualidade de Software: Fundamento Essencial

- Em um mercado competitivo, a **qualidade** é um fator determinante para o sucesso de qualquer produto, incluindo o software.
- Software de alta qualidade atende aos requisitos, é confiável, eficiente, usável e manutenível.
- Garantir a qualidade do software é um processo contínuo e multifacetado, onde o **teste de software** desempenha um papel crucial.

Teste de Software: Uma Definição Abrangente

- O **teste de software** é um processo sistemático para avaliar a qualidade de um software, identificando defeitos (bugs) e verificando se ele atende aos requisitos especificados e às expectativas do usuário.
- Envolve a execução do software sob condições controladas, analisando o comportamento e os resultados obtidos.
- Não se limita a encontrar erros, mas também visa garantir a confiabilidade, o desempenho e a **usabilidade** do sistema.

Objetivos Primários do Teste de Software

- **Identificação de Defeitos:** Descobrir erros, falhas e inconsistências no software antes que ele seja entregue aos usuários finais.
- **Verificação e Validação (V&V):**
 - ▶ **Verificação:** Garantir que o software está sendo construído *corretamente* ("Estamos fazendo a coisa do jeito certo?").
 - ▶ **Validação:** Garantir que o software construído atende às *necessidades reais* dos stakeholders ("Estamos fazendo a coisa certa?").
- **Garantia da Qualidade:** Fornecer informações aos stakeholders sobre o nível de qualidade do software.

O Teste como um Processo, Não um Evento Isolado

- O teste de software não é uma atividade pontual ao final do desenvolvimento.
- É um **processo contínuo** que deve ser integrado em todas as fases do ciclo de vida do desenvolvimento de software (SDLC).
- Envolve planejamento, projeto de casos de teste, execução, análise de resultados e relatórios.
- A participação de diferentes membros da equipe (testadores, desenvolvedores, analistas) é fundamental.

Casos de Teste: A Unidade Fundamental do Teste

- Um **caso de teste** é um conjunto de condições, dados de entrada, ações de execução e resultados esperados, desenvolvido para verificar um requisito específico ou uma funcionalidade do software.
- Elementos de um caso de teste:
 - ▶ ID único
 - ▶ Descrição do objetivo
 - ▶ Pré-condições
 - ▶ Dados de entrada
 - ▶ Passos de execução
 - ▶ Resultado esperado
- A qualidade dos casos de teste impacta diretamente a eficácia do processo de teste.

Validação vs. Verificação: Foco e Momento

- **Verificação** concentra-se no processo de desenvolvimento e nos artefatos criados (documentação, código). Ocorre durante o desenvolvimento.
- **Validação** concentra-se no produto final e em sua adequação às necessidades do usuário. Ocorre principalmente antes da entrega ao cliente.
- Ambas são atividades essenciais para garantir um software de qualidade, mas com focos distintos.

A Importância do Planejamento de Testes

- Um **plano de testes** é um documento que descreve o escopo, os objetivos, a abordagem, os recursos e o cronograma das atividades de teste.
- Elementos chave de um plano de testes:
 - ▶ Definição do escopo e dos objetivos do teste.
 - ▶ Estratégia de teste (tipos de teste a serem realizados).
 - ▶ Recursos necessários (equipe, ferramentas, ambiente).
 - ▶ Cronograma e prazos.
 - ▶ Critérios de entrada e saída do teste.
 - ▶ Gerenciamento de riscos.
- Um plano bem elaborado é crucial para a eficiência e eficácia do processo de teste.

Níveis de Teste: Uma Abordagem Estruturada

- Os testes de software são frequentemente organizados em diferentes níveis, cada um com um foco específico:
 - ▶ **Teste de Unidade:** Testa componentes individuais do software (funções, métodos, classes).
 - ▶ **Teste de Integração:** Testa a interação entre diferentes unidades ou módulos do software.
 - ▶ **Teste de Sistema:** Testa o sistema como um todo, verificando a conformidade com os requisitos funcionais e não funcionais.
 - ▶ **Teste de Aceitação:** Realizado pelos usuários finais ou clientes para determinar se o software atende às suas necessidades e está pronto para implantação.

Tipos de Teste: Explorando Diferentes Perspectivas

- Existem diversos tipos de teste, classificados de acordo com diferentes critérios:
 - ▶ **Teste Funcional:** Verifica se o software se comporta conforme as funcionalidades especificadas.
 - ▶ **Teste Não Funcional:** Avalia aspectos como desempenho, segurança, usabilidade e confiabilidade.
 - ▶ **Teste de Caixa Branca (Estrutural):** Baseado na estrutura interna do código.
 - ▶ **Teste de Caixa Preta (Comportamental):** Baseado nos requisitos e funcionalidades externas.
- A escolha dos tipos de teste depende das características do software e dos objetivos do teste.

A Mentalidade do Testador: Um Olhar Crítico

- Um bom testador possui uma mentalidade crítica e busca ativamente por defeitos.
- Questiona as suposições, explora diferentes cenários e pensa "fora da caixa".
- É detalhista, organizado, comunicativo e persistente.
- Compreende a perspectiva do usuário final e busca garantir uma experiência de qualidade.

Depuração: Rastreando e Corrigindo Defeitos

- A **depuração** é o processo de identificar, isolar e corrigir os defeitos (bugs) encontrados durante o teste de software.
- Envolve a análise do código-fonte, a execução passo a passo do programa e a inspeção do estado das variáveis para entender a causa da falha.
- Ferramentas de depuração (debuggers) em IDEs (Ambientes de Desenvolvimento Integrados) facilitam esse processo.
- A colaboração entre testadores e desenvolvedores é crucial para uma depuração eficiente.

Automação de Testes: Aumentando a Eficiência

- A **automação de testes** envolve o uso de ferramentas de software para executar casos de teste predefinidos, comparar os resultados obtidos com os resultados esperados e gerar relatórios de teste.
- Ideal para testes repetitivos, testes de regressão e testes de desempenho.
- Aumenta a velocidade, a cobertura e a consistência dos testes, liberando os testadores para tarefas mais complexas e exploratórias.
- A escolha das ferramentas de automação e a criação de scripts de teste eficazes são aspectos importantes da automação.

Gerenciamento de Testes: Organização e Controle

- O **gerenciamento de testes** abrange as atividades de planejamento, organização, execução, monitoramento e controle do processo de teste.
- Envolve a gestão de casos de teste, planos de teste, dados de teste, ambientes de teste e relatórios de teste.
- Ferramentas de gerenciamento de testes auxiliam na organização e no rastreamento das atividades de teste, facilitando a comunicação entre a equipe.
- Métricas de teste (cobertura, taxa de defeitos, tempo de execução) são importantes para monitorar o progresso e a qualidade do teste.

Desenvolvimento Orientado a Testes (TDD): Testando Antes de Codificar

- O **Desenvolvimento Orientado a Testes (TDD)** é uma prática de desenvolvimento de software onde os casos de teste são escritos *antes* da implementação do código.
- Segue um ciclo de "Red-Green-Refactor":
 - ▶ **Red:** Escrever um caso de teste que inicialmente falha (pois o código ainda não existe).
 - ▶ **Green:** Escrever o código mínimo necessário para fazer o teste passar.
 - ▶ **Refactor:** Refatorar o código para melhorar sua estrutura e legibilidade, garantindo que os testes continuem passando.

Desenvolvimento Orientado a Testes (TDD): Testando Antes de Codificar

TDD ajuda a garantir que o código seja testável, bem estruturado e atenda aos requisitos desde o início.

Cobertura de Teste: Medindo a Extensão dos Testes

- A **cobertura de teste** é uma métrica que indica a proporção do software que foi exercitada pelos casos de teste.
- Diferentes tipos de cobertura:
 - ▶ Cobertura de código (declarações, branches, condições).
 - ▶ Cobertura funcional (requisitos, funcionalidades).
- Uma alta cobertura não garante a ausência de defeitos, mas aumenta a confiança de que as partes **testadas** do software **funcionam corretamente**.

Testes de Regressão: Garantindo a Estabilidade Após Mudanças

- Os **testes de regressão** são realizados após a implementação de correções de defeitos, novas funcionalidades ou outras mudanças no software.
- O objetivo é garantir que as alterações não introduziram novos defeitos em outras partes do sistema e que as funcionalidades existentes continuam funcionando corretamente.
- A automação é particularmente útil para testes de regressão devido à sua natureza repetitiva.

Ambiente de Teste: Simulando o Mundo Real

- O **ambiente de teste** é a configuração de hardware, software e rede onde os testes são executados.
- Idealmente, o ambiente de teste deve ser o mais semelhante possível ao ambiente de produção para garantir a relevância dos resultados dos testes.
- A configuração e o gerenciamento adequados do ambiente de teste são essenciais para a execução eficaz dos testes.

Dados de Teste: Entradas Significativas

- Os **dados de teste** são os valores de entrada utilizados para executar os casos de teste.
- A escolha de dados de teste relevantes e representativos é crucial para cobrir diferentes cenários e identificar possíveis problemas.
- Técnicas de projeto de dados de teste (análise de valor limite, particionamento de equivalência) ajudam a criar conjuntos de dados eficazes.

Relatórios de Teste: Comunicando os Resultados

- Os **relatórios de teste** documentam as atividades de teste, os resultados obtidos, os defeitos encontrados e o status geral da qualidade do software.
- Fornecem informações importantes para os stakeholders tomarem decisões sobre a liberação do software.
- Um bom relatório de teste é claro, conciso, objetivo e inclui informações relevantes, como métricas de teste, resumo dos defeitos e recomendações.

A Evolução Contínua do Teste de Software

- O campo do teste de software está em constante evolução, com novas metodologias, ferramentas e técnicas surgindo regularmente.
- Abordagens como testes exploratórios, testes baseados em risco e inteligência artificial aplicada a testes estão ganhando importância.
- Manter-se atualizado com as últimas tendências e melhores práticas é essencial para os profissionais de teste.

Testes em Diferentes Paradigmas: Mobile e Orientado a Objetos

- **Testes para Aplicações Móveis:** Consideram a diversidade de dispositivos, sistemas operacionais (Android, iOS), tamanhos de tela, conectividade e aspectos de usabilidade específicos para o contexto mobile.
- **Testes em Aplicações Orientadas a Objetos:** Focam em conceitos como classes, objetos, herança, polimorfismo e encapsulamento, exigindo estratégias de teste que validem as interações e o comportamento desses elementos.

Testes de Funcionalidade: Validando o Comportamento do Software

- Os **testes de funcionalidade** verificam se cada funcionalidade do software opera de acordo com os requisitos especificados.
- Envolve a criação de casos de teste que cobrem todas as entradas possíveis, o processamento e as saídas esperadas para cada função.
- Técnicas como análise de valor limite e particionamento de equivalência são frequentemente utilizadas no projeto de testes funcionais.

Testes Estruturais (Caixa Branca): Olhando para Dentro do Código

- Os **testes estruturais**, também conhecidos como testes de caixa branca, examinam a estrutura interna do código-fonte.
- O objetivo é garantir que todos os caminhos de execução, decisões lógicas e laços sejam testados adequadamente.
- Métricas de cobertura de código (cobertura de declaração, cobertura de decisão, cobertura de caminho) são usadas para avaliar a eficácia dos testes estruturais.

Testes Não Funcionais: Além do Comportamento

- Os **testes não funcionais** avaliam aspectos da qualidade do software que não estão diretamente relacionados às suas funcionalidades.
- Exemplos incluem:
 - ▶ **Teste de Desempenho:** Avalia a velocidade, a capacidade de resposta e a estabilidade do software sob diferentes cargas.
 - ▶ **Teste de Segurança:** Identifica vulnerabilidades e garante a proteção contra acesso não autorizado.
 - ▶ **Teste de Usabilidade:** Avalia a facilidade com que os usuários podem interagir com o software.
 - ▶ **Teste de Confiabilidade:** Mede a probabilidade de o software operar sem falhas por um período especificado.

A Mentalidade do Testador: Curiosidade e Atenção aos Detalhes

- Um testador eficaz possui uma curiosidade natural para explorar o software e identificar comportamentos inesperados.
- A atenção aos detalhes é fundamental para notar inconsistências, erros de digitação e pequenas falhas que podem indicar problemas maiores.
- A capacidade de se colocar no lugar do usuário final ajuda a identificar problemas de usabilidade e fluxo.

Comunicação Eficaz no Processo de Teste

- A comunicação clara e concisa é essencial entre os membros da equipe de teste, desenvolvedores e outros stakeholders.
- Relatórios de defeitos bem escritos, com passos claros para reprodução e informações relevantes, facilitam a correção dos problemas.
- Reuniões regulares de status e discussões sobre os resultados dos testes mantêm todos informados sobre o progresso e os desafios.

A Importância da Ferramentas de Teste

- Diversas ferramentas auxiliam o processo de teste, desde o gerenciamento de casos de teste até a automação e o monitoramento de desempenho.
- A escolha das ferramentas adequadas depende das necessidades específicas do projeto e da expertise da equipe.
- O uso eficiente de ferramentas pode aumentar significativamente a produtividade e a eficácia do teste.

Testes Exploratórios: Liberdade e Descoberta

- O **teste exploratório** é uma abordagem de teste menos formal, onde o testador simultaneamente aprende sobre o software, projeta e executa testes.
- Enfatiza a liberdade, a adaptabilidade e a criatividade do testador.
- Útil para descobrir problemas inesperados e em áreas do software menos documentadas ou compreendidas.

Testes Baseados em Risco: Priorizando o Esforço

- Os **testes baseados em risco** concentram os esforços de teste nas áreas do software que apresentam maior probabilidade de falha e maior impacto nos negócios.
- A análise de riscos ajuda a identificar essas áreas críticas e a alocar os recursos de teste de forma mais eficiente.
- Permite garantir a qualidade nas áreas mais importantes do sistema, mesmo com recursos limitados.

Testes de Usabilidade: A Experiência do Usuário em Foco

- Os **testes de usabilidade** avaliam a facilidade com que os usuários podem aprender, operar, entender e se sentir satisfeitos ao interagir com o software.
- Envolve a observação de usuários reais realizando tarefas específicas e a coleta de feedback sobre sua experiência.
- Identifica problemas de design, fluxo de navegação e outros aspectos que podem impactar a satisfação do usuário.

Testes de Desempenho: Garantindo a Resposta e a Escalabilidade

- Os **testes de desempenho** medem a capacidade de resposta, a estabilidade e o consumo de recursos do software sob diferentes condições de carga.
- Incluem testes de carga (simulando o uso normal), testes de estresse (levando o sistema ao limite) e testes de pico (simulando aumentos repentinos de usuários).
- Garantem que o software possa lidar com o número esperado de usuários e dados sem degradação **significativa** no desempenho.

Testes de Segurança: Protegendo Dados e Funcionalidades

- Os **testes de segurança** visam identificar vulnerabilidades no software que possam ser exploradas por atacantes.
- Incluem testes de penetração, análise de vulnerabilidades, testes de autenticação e autorização, e testes de segurança de dados.
- Essenciais para proteger informações confidenciais e garantir a integridade do sistema.

Testes de Confiabilidade: Avaliando a Estabilidade a Longo Prazo

- Os **testes de confiabilidade** avaliam a probabilidade de o software operar sem falhas por um período específico em um ambiente determinado.
- Envolve a execução do software por longos períodos sob condições normais e a observação de eventuais falhas ou degradação no desempenho.
- Importantes para sistemas críticos onde a **disponibilidade** contínua é essencial.

A Importância da Documentação de Teste

- Uma **documentação de teste** clara e completa é fundamental para o sucesso do processo de teste.
- Inclui planos de teste, casos de teste, dados de teste, relatórios de defeitos, relatórios de resumo de teste e outros documentos relevantes.
- Facilita a comunicação, o rastreamento, a replicabilidade e a melhoria contínua do processo de teste.

Métricas de Teste: Medindo o Progresso e a Qualidade

- As **métricas de teste** fornecem informações quantitativas sobre o processo de teste e a qualidade do software.
- Exemplos incluem número de casos de teste executados, taxa de aprovação, número de defeitos encontrados, taxa de defeitos por severidade, cobertura de código e tempo de ciclo de teste.
- Permitem monitorar o progresso, identificar áreas de melhoria e tomar decisões informadas sobre a liberação do software.

O Papel da Automação na Agilidade

- Em metodologias ágeis, a **automação de testes** desempenha um papel crucial para garantir feedback rápido e contínuo sobre a qualidade do software.
- Testes automatizados permitem que a equipe valide as mudanças de código com frequência, facilitando a integração contínua e a entrega contínua (CI/CD).
- Reduz o tempo de ciclo de desenvolvimento e aumenta a confiança na qualidade das entregas.

A Colaboração entre Testadores e Desenvolvedores

- Uma forte **colaboração** entre testadores e desenvolvedores é essencial para um processo de teste eficaz.
- A comunicação aberta, o compartilhamento de conhecimento e o trabalho em conjunto para identificar e corrigir defeitos levam a um software de maior qualidade.
- A integração dos testes desde as fases iniciais do desenvolvimento (Shift Left) promove a prevenção de defeitos.

A Melhoria Contínua do Processo de Teste

- O processo de teste deve ser continuamente avaliado e aprimorado.
- A análise das métricas de teste, a realização de retrospectivas e a adoção de novas técnicas e ferramentas contribuem para aumentar a eficiência e a eficácia do teste ao longo do tempo.
- Aprender com os erros e adaptar as estratégias de teste são fundamentais para a maturidade do processo.

O que é Planejamento da Auditoria de Software?

Definição

O planejamento da auditoria de software é a fase inicial e crucial onde definimos o **escopo** (o que será auditado), os **objetivos** (o que esperamos alcançar), a **equipe** (quem fará a auditoria) e os **recursos necessários** (tempo, ferramentas, etc.) para conduzir uma auditoria de software eficaz e com resultados significativos.

O que é Planejamento da Auditoria de Software?

- **Fundamental para o sucesso:** Garante que a auditoria seja eficiente, focada e alcance seus objetivos.
- **Direciona a auditoria:** Define a abordagem estratégica, o cronograma realista e a metodologia a ser utilizada.
- **Base para as próximas fases:** Um bom planejamento facilita a execução, a análise e a geração de relatórios da auditoria.

Por que o Planejamento é Tão Importante?

- **Minimiza Riscos:** Reduz a probabilidade de auditorias mal executadas, superficiais ou que não agregam valor.
- **Otimiza Recursos:** Evita o desperdício de tempo, dinheiro e esforço, direcionando os recursos para as áreas mais relevantes.
- **Melhora a Comunicação:** Promove o entendimento e a colaboração entre a equipe de auditoria, a gestão e as áreas auditadas.
- **Foco nos Pontos Críticos:** Permite identificar e priorizar a análise das áreas mais sensíveis e com maior potencial de risco no sistema.
- **Garante a Objetividade:** Ajuda a manter o foco nos objetivos definidos, evitando desvios e garantindo uma avaliação imparcial.

Etapas Essenciais do Planejamento

- 1 Definição Clara do Escopo e Objetivos:** Delimitar o que será auditado e quais resultados concretos são esperados.
- 2 Identificação Detalhada dos Sistemas a Serem Auditados:** Listar e descrever os sistemas, módulos ou componentes que serão foco da auditoria.
- 3 Avaliação Preliminar de Riscos e Controles Existentes:** Identificar potenciais vulnerabilidades e os mecanismos de controle já implementados.

Etapas Essenciais do Planejamento

- 4 **Escolha da Metodologia e Técnicas de Auditoria Adequadas:** Selecionar as abordagens (ex: baseada em riscos, conformidade) e as técnicas (ex: entrevistas, análise documental, testes) mais apropriadas.
- 5 **Alocação Estratégica de Recursos e Formação da Equipe:** Designar os membros da equipe com as habilidades necessárias e alocar o tempo e o orçamento disponíveis.
- 6 **Elaboração de um Cronograma Detalhado e Realista:** Definir prazos para cada etapa do processo de auditoria.

Detalhando o Escopo e os Objetivos da Auditoria

- **O Que Será Auditado? (Delimitação):** Qual sistema, módulo, processo ou área específica será o foco da auditoria?
- **Qual o Nível de Profundidade? (Granularidade):** Quão detalhada será a análise? Serão revisados códigos, logs, configurações, processos de desenvolvimento?
- **Que Resultados Concretos São Esperados? (Entregáveis):** Quais relatórios, recomendações ou planos de ação devem ser gerados ao final da auditoria?
- **Quem São os Stakeholders e Quais São Suas Expectativas? (Partes Interessadas):** Quem se beneficiará ou será impactado pela auditoria? Quais são suas principais preocupações?

Exemplo Prático de Definição de Escopo

Sistema: Aplicação Web de Vendas (Foco em Segurança e Conformidade)

- **Segurança de Dados dos Clientes:** Verificar a robustez das medidas de proteção de informações pessoais (cadastros, dados de pagamento).
- **Conformidade com a LGPD (Lei Geral de Proteção de Dados):** Avaliar se a aplicação coleta, armazena e processa dados de acordo com os requisitos legais.
- **Rastreabilidade e Auditoria de Acessos (Logs):** Analisar os logs de acesso para identificar atividades suspeitas e garantir a rastreabilidade das ações dos usuários.
- **Segurança na Autenticação e Autorização:** Verificar se os mecanismos de login e controle de acesso são seguros e impedem

A Importância da Equipe de Auditoria

- **Analistas de Sistemas:** Essenciais para entender a arquitetura, o funcionamento e os processos do sistema.
- **Especialistas em Segurança da Informação:** Cruciais para avaliar vulnerabilidades, testes de intrusão e conformidade com padrões de segurança.
- **Auditores Internos ou Externos:** Trazem a perspectiva de controle, conformidade e melhores práticas de auditoria.
- **Responsável Técnico pelo Sistema (Participação):** Fornece conhecimento detalhado do sistema e facilita o entendimento da equipe de auditoria.

A Importância da Equipe de Auditoria

CrITÉRIOS de Seleção da Equipe

A escolha dos membros da equipe deve ser baseada em **experiência** comprovada, **conhecimento técnico** específico das áreas a serem auditadas e, fundamentalmente, **imparcialidade** para garantir uma avaliação objetiva.

Ferramentas e Técnicas Essenciais na Auditoria

- **Checklists de Conformidade:** Listas de verificação baseadas em normas, padrões ou requisitos legais para garantir a aderência.
- **Análise Estática de Código-Fonte:** Ferramentas que analisam o código sem executá-lo, buscando por vulnerabilidades, erros de programação e desvios de padrões.
- **Testes de Penetração (Dinâmica):** Simulações de ataques para identificar falhas de segurança exploráveis em um ambiente controlado.

Ferramentas e Técnicas Essenciais na Auditoria

■ Ferramentas de Auditoria Especializadas:

- ▶ **Wireshark:** Análise de tráfego de rede para identificar vulnerabilidades e problemas de comunicação.
- ▶ **OWASP ZAP (Zed Attack Proxy):** Ferramenta para testes de segurança em aplicações web.
- ▶ **Nessus/OpenVAS:** Scanners de vulnerabilidades para identificar falhas de segurança em sistemas e redes.
- ▶ **Ferramentas de Análise de Logs:** Para identificar padrões de acesso, atividades suspeitas e problemas de desempenho.

Boas Práticas para um Planejamento Eficaz

- **Definir Metas Claras e Mensuráveis (SMART):** As metas devem ser Específicas, Mensuráveis, Atingíveis, Relevantes e com Prazos definidos.
- **Estabelecer Canais de Comunicação Eficazes:** Manter as áreas auditadas informadas sobre o processo e os achados da auditoria.
- **Documentar Detalhadamente Todo o Processo de Planejamento:** Registrar as decisões, o escopo, os objetivos, a metodologia e o cronograma.

Boas Práticas para um Planejamento Eficaz

- **Garantir a Confidencialidade e a Integridade das Informações Coletadas:** Proteger os dados acessados durante a auditoria.
- **Obter o Apoio da Alta Gestão:** O comprometimento da liderança é crucial para o sucesso da auditoria.
- **Ser Flexível e Adaptável:** Estar preparado para ajustar o plano conforme novas informações ou desafios surgirem.

Situação-Problema: Auditoria em Sistema de Gestão Hospitalar

Você foi contratado como auditor para avaliar um sistema de gestão hospitalar que armazena dados sensíveis de pacientes, gerencia agendamentos, prontuários eletrônicos e informações financeiras. Elabore um escopo preliminar com base em possíveis riscos e objetivos da auditoria.

Situação-Problema: Auditoria em Sistema de Gestão Hospitalar

Você foi contratado como auditor para avaliar um sistema de gestão hospitalar que armazena dados sensíveis de pacientes, gerencia agendamentos, prontuários eletrônicos e informações financeiras. Elabore um escopo preliminar com base em possíveis riscos e objetivos da auditoria.

- **Quais áreas específicas do sistema você priorizaria para investigar e por quê?**
- **Que tipos de ferramentas e técnicas de auditoria você consideraria mais relevantes para este cenário?**
- **Como você estruturaria uma pequena equipe de auditoria para cobrir as áreas identificadas?**

Controles Gerais de Auditoria de Sistemas

- **Definição:** Estruturas internas, políticas administrativas e procedimentos operacionais estabelecidos nas empresas.
- **Importância:** Fundamentais para orientar o desenvolvimento, a implementação e a auditoria de sistemas de informação.
- **Impacto:** Uma auditoria eficaz contribui significativamente para a qualidade dos processos e produtos de software.

A Centralidade dos Controles Gerais

- **Moldam o Ambiente Organizacional:** Influenciam a cultura de segurança, a gestão de riscos e a forma como os sistemas são desenvolvidos e utilizados.
- **Influenciam a Abordagem da Auditoria:** A robustez dos controles gerais pode impactar a profundidade e o foco da auditoria de sistemas.
- **Exemplo Prático:** Restrições bem definidas sobre quem pode conceder descontos no sistema de vendas (controle de acesso e autorização) facilitam a auditoria dessa funcionalidade.

Conceito-Chave (Braz, 2017)

Definição

Estruturas internas, políticas administrativas e procedimentos utilizados em todas as atividades operacionais de uma organização.

- **Objetivo:** Controlar as regras de negócio, definir responsabilidades e garantir a conformidade com as políticas internas e externas.
- **Relevância para Auditoria:** São a base para avaliar a eficácia dos controles específicos dos sistemas e identificar vulnerabilidades na governança de TI.

Categorias de Controles Gerais

Categoria		Descrição
Controle Organizacional	Organi-	Políticas e procedimentos formais, estrutura organizacional clara, segregação de funções e delegação de responsabilidades.
Controle Geral de Segurança	Geral de	Políticas de segurança da informação, gestão de riscos de TI, supervisão das atividades de segurança e conscientização dos usuários.
Continuidade de Serviço	de	Planos de recuperação de desastres (DRP) e planos de continuidade de negócios (BCP) para garantir a disponibilidade dos sistemas em caso de falhas ou incidentes.

Categorias de Controles Gerais

Categoria	Descrição
Controle de Software de Sistema	Políticas e procedimentos para gerenciar o acesso a arquivos críticos do sistema operacional e outros softwares de base.
Controle de Acesso	Políticas e mecanismos (senhas, biometria, cartões) para evitar acessos não autorizados a sistemas, dados e instalações.
Controle de Versionamento	Processos para gerenciar e rastrear as modificações e diferentes versões dos sistemas e aplicações.

Controle de Software de Sistema

Definição: Conjunto de programas essenciais que gerenciam e controlam as atividades de processamento de um computador.

- **Sistemas Operacionais:** Windows, Linux, macOS (gerenciam recursos e acesso).
- **Utilitários de Segurança:** Firewalls, antivírus, sistemas de detecção de intrusão (protegem o sistema).
- **Bibliotecas de Software:** Conjuntos de código reutilizável (podem conter vulnerabilidades).
- **Software de Comunicação:** Protocolos e aplicações de rede (podem ser vetores de ataque).
- **Sistemas de Gerenciamento de Banco de Dados (SGBD):** MySQL, PostgreSQL, Oracle (armazenam dados sensíveis).

Sistemas Relacionados a um Sistema de Caixa (Exemplo)

- **Sistema Operacional:** Plataforma base (ex: Windows para PDV).
- **Utilitários de Segurança:** Antivírus local, firewall.
- **Sistemas de Comunicação:** Drivers de rede, software de comunicação com servidor central (ex: para envio de vendas).
- **Sistema de Gerenciamento de Banco de Dados (se aplicável):** Para armazenar dados de vendas locais temporariamente.

Pontos Críticos de Auditoria no Software de Sistema

- **Controle de Acesso ao Software de Sistema:** Quem tem permissão para instalar, configurar ou modificar o sistema operacional e outros softwares de base?
- **Supervisão e Monitoramento do Software de Sistema:** Existem logs de eventos de segurança? As atividades dos administradores são monitoradas?
- **Controle de Alterações no Software de Sistema:** Como as atualizações e patches são aplicados? Existe um processo formal de gestão de mudanças?

Controle de Aplicativos

- **Foco na Funcionalidade:** As aplicações de software realizam tarefas específicas para os usuários.
- **Três Funções Básicas:**
 - ▶ **Entrada de Dados:** Como os dados são inseridos no sistema (formulários, arquivos, etc.)? Existem controles para garantir a validade e integridade?
 - ▶ **Processamento de Dados:** Como os dados são transformados e manipulados pela aplicação? Os algoritmos são seguros e confiáveis?
 - ▶ **Saída de Dados/Informações:** Como os resultados são apresentados aos usuários (relatórios, telas, etc.)? As informações confidenciais são protegidas?

Controle de Aplicativos

Objetivo da Auditoria de Aplicativos: Garantir a **integridade** (dados corretos e completos) e a **confiabilidade** (sistema funcionando conforme esperado) em cada etapa do processamento.

Necessidade de Evolução e Manutenção

- **Ciclo de Vida Dinâmico:** Os processos de evolução e manutenção acompanham todo o ciclo de vida do software, desde a concepção até a obsolescência.
- **Evolução Além da Correção:** Não se limita a corrigir falhas, mas também envolve a adaptação a novos requisitos de negócios, tecnologias emergentes e expectativas dos usuários.
- **Fundamental para a Longevidade:** A evolução contínua é essencial para manter o software relevante, competitivo e com qualidade ao longo do tempo.

A Importância da Manutenção e Evolução

- **Realidade do Dia a Dia:** Manutenção e evolução são atividades predominantes na rotina de muitos desenvolvedores e engenheiros de software.
- **Conhecimento Essencial:** Compreender as características, os desafios e as melhores práticas da manutenção é fundamental para o sucesso na carreira.
- **Objetivo Profissional:** Contribuir para garantir a qualidade, a conformidade, a segurança e a adaptação contínua dos sistemas de software.

Sobre o Envelhecimento do Software

- **Obsolescência de Sistemas Proprietários:** Podem ser descontinuados pelo fornecedor, deixando de receber atualizações de segurança e novas funcionalidades.
- **Vantagens do Software Livre:** Geralmente possuem suporte ativo de uma comunidade de desenvolvedores, garantindo atualizações e correções.
- **O Envelhecimento é Inevitável:** Todo software, mesmo bem construído, está sujeito ao envelhecimento devido a mudanças tecnológicas, requisitos de negócios e expectativas dos usuários.
- **Impacto na Continuidade:** O processo de envelhecimento, se não gerenciado, pode comprometer a continuidade e a capacidade de evolução do sistema.

Tipos de Envelhecimento do Software

Falha de Adequação

Ocorre quando o software não atende mais às necessidades atuais dos usuários ou da organização, seja por erros conceituais no desenvolvimento inicial ou pela falta de adaptação a novas demandas, perdendo assim sua integridade e confiabilidade do ponto de vista funcional.

Falha na Mudança

Resulta de atualizações ou modificações mal planejadas ou implementadas, que introduzem novos defeitos ou impactam negativamente funcionalidades existentes, comprometendo a estabilidade e a qualidade do software.

Exemplos Práticos de Envelhecimento

- **Falha de Adequação:** Um software de conversão de vídeo que não suporta os formatos mais recentes ou não funciona corretamente na nova versão do sistema operacional.
- **Falha na Mudança:** A implementação incorreta de uma nova API de pagamento em um sistema de e-commerce que causa erros no processamento das transações ou expõe dados sensíveis dos clientes.

Classificação e Tipos de Manutenção

Classificação	Conceito	Exemplo
Adaptativa	Ajustes para atender a novos requisitos do ambiente (hardware, SO, regulamentações).	Implantação do sistema de pagamento PIX em aplicativos de bancos.
Corretiva	Correção de defeitos (bugs) identificados no software.	Ajustes emergenciais no aplicativo e-Título durante o período eleitoral.
Evolutiva	Adição de novas funcionalidades e melhorias no sistema.	Implementação de um chat com inteligência artificial em um e-commerce para suporte ao cliente.

A Relevância da Classificação da Manutenção

- **Melhor Compreensão das Necessidades:** Ajuda desenvolvedores, gestores e clientes a identificar claramente o tipo de intervenção necessária no software.
- **Alocação Eficiente de Recursos:** Facilita o planejamento e a alocação adequada de tempo, orçamento e equipe para cada tipo de manutenção durante o ciclo de vida do software.
- **Aumento da Assertividade:** Melhora a precisão no diagnóstico de problemas (manutenção corretiva) e na definição de escopo de novas funcionalidades (manutenção evolutiva).

Técnicas e Ferramentas Essenciais na Manutenção

- **Objetivo:** Garantir a aplicação de boas práticas de desenvolvimento e a conformidade com os requisitos durante as atividades de manutenção.
- **Exemplos de Boas Práticas:**
 - ▶ **Codificação Legível:** Utilizar nomes significativos para variáveis e funções, adicionar comentários explicativos e manter a indentação consistente.
 - ▶ **Versionamento de Código:** Utilizar sistemas de controle de versão (como Git) para rastrear e gerenciar as alterações no código-fonte.

Boas Práticas de Codificação: Clareza e Organização

- **Nomes Descritivos:** Escolher nomes de variáveis e funções que indiquem claramente seu propósito.
- **Comentários Significativos:** Adicionar comentários para explicar partes complexas do código ou a lógica por trás de certas implementações.
- **Indentação Consistente:** Utilizar a indentação para representar a estrutura hierárquica do código, facilitando a leitura e a compreensão.

Boas Práticas de Codificação: Clareza e Organização

```
// Função para limpar os campos do formulário de endereço
function limpaFormularioCep() {
    // Limpa os valores dos elementos do formulário
    document.getElementById('rua').value = "";
    document.getElementById('bairro').value = "";
    document.getElementById('cidade').value = "";
    document.getElementById('estado').value = "";
}
```

O Versionamento de Software tem como propósito documentar todas as modificações e atualizações realizadas no software ao longo do tempo.

■ **Sistema de Numeração de Versões (Ex Semântico: 10.2.4.3):**

- ▶ **10 (Major):** Mudanças significativas na arquitetura ou funcionalidades principais que podem quebrar a compatibilidade com versões anteriores.
- ▶ **2 (Minor):** Adição de novas funcionalidades que são compatíveis com a versão anterior.
- ▶ **4 (Patch):** Correções de bugs e pequenas melhorias que não introduzem novas funcionalidades.
- ▶ **3 (Build/Hotfix - Opcional):** Correções de segurança urgentes ou builds específicos.

■ **Mensagens de Commit:** Adicionar comentários claros e concisos ao registrar as alterações no sistema de versionamento para facilitar a compreensão das modificações futuras.